

Assahou Emmanuel

Dabout Mattéo

Big Data

Oral Projet

Partie Business :

Bonjour à tous. Aujourd'hui, nous allons vous plonger au cœur de la microstructure des marchés financiers. Un univers où les décisions ne se prennent plus en jours ou en heures, mais à la microseconde.

Notre projet porte sur la conception d'un moteur « RegTech » — c'est-à-dire une technologie au service de la régulation — dédié à la surveillance des marchés à haute fréquence. Le sous-jacent que nous avons choisi de modéliser est d'une actualité stratégique : le marché des quotas carbone.

Sur ce marché dématérialisé, la liquidité est vitale. Les industriels et les fonds d'investissement y échangent des droits d'émission de CO2 en flux continu. Mais cette course à la liquidité et à la vitesse a ouvert la porte à une forme de criminalité financière algorithmique particulièrement redoutable : la manipulation de carnet d'ordres, et plus spécifiquement, le *Spoofing*.

Qu'est-ce que le spoofing ? Pour le dire simplement, c'est l'art de créer un mirage financier pour pirater la perception des autres algorithmes.

Concrètement, un acteur malveillant va injecter un volume colossal de faux ordres d'achat d'un côté du carnet, sans *aucune* intention de les exécuter. Son seul but est de créer ce que l'on appelle un *Order Imbalance*, un déséquilibre artificiel de l'offre et de la demande.

En voyant apparaître ce mur d'achat géant, les algorithmes de trading adverses (comme les Market Makers) vont réagir. Ils vont anticiper une hausse imminente du prix du quota carbone et se ruer pour acheter. C'est à ce moment précis que le spoofer — qui avait discrètement placé de vrais ordres de vente de l'autre côté du carnet — réalise son profit. En quelques millisecondes, il vend ses actifs à un prix artificiellement gonflé, puis annule instantanément son faux mur d'achat. Le mirage se dissipe, la demande s'évapore, le prix s'effondre. Le marché a été dupé.

Ce n'est pas qu'un simple problème théorique. Le spoofing est un délit extrêmement dangereux, car il détruit la confiance et l'intégrité même du marché. Il assèche la véritable liquidité, fausse le processus de découverte des prix, et peut déclencher des *Flash Crashes* dévastateurs qui ruinent les acteurs légitimes.

C'est pourquoi les gendarmes financiers, comme l'AMF en France ou la SEC aux États-Unis, ont l'obligation légale et absolue — notamment dictée par la directive européenne MiFID II — de repérer et de sanctionner ces manipulations.

Cependant, intercepter un spoofer dans la vraie vie est un véritable cauchemar analytique pour les régulateurs. L'AMF doit littéralement trouver une aiguille dans une botte de foin, où la botte de foin représente des dizaines de milliers de microtransactions par seconde, et où le fraudeur utilise des techniques stochastiques pour se fondre dans le bruit des acteurs sains.

C'est très exactement ce défi hybride que nous avons choisi de relever aujourd'hui : concevoir l'architecture Big Data capable d'encaisser cette vélocité extrême, tout en appliquant une modélisation mathématique probabiliste pour démasquer la fraude en temps réel.

Partie Problèmes et Solutions :

Maintenant que l'enjeu métier est posé, parlons du mur technologique auquel nous nous sommes heurtés. Surveiller un carnet d'ordres à haute fréquence soulève un problème d'ingénierie majeur : la gestion des *Heavy Writes*, ou la tempête d'écritures.

Imaginons que nous utilisions une architecture classique avec une base de données relationnelle SQL pour traiter et surveiller les ordres. Dans un système SQL standard, pour garantir l'intégrité financière, la base verrouille la donnée lors d'une transaction. Si un spoofer injecte des centaines d'ordres d'un coup pour créer son leurre, la base de données va tenter de les traiter un par un, provoquant un goulet d'étranglement fatal. Le serveur s'étouffe, le carnet d'ordres fige, et le marché s'arrête.

La solution algorithmique que nous avons implémentée repose sur deux piliers du Big Data : l'asynchronisme et le découplage strict entre l'opérationnel (OLTP) et l'analytique (OLAP).

Au lieu d'écrire directement sur le disque dur, notre moteur de marché FastAPI encaisse le choc en mémoire vive (RAM) via le motif *Fan-In*. Les requêtes de trading tombent instantanément dans une file d'attente asynchrone non-bloquante (asyncio.Queue). L'API accuse réception à la milliseconde près.

Ensuite, nous séparons les flux : Le chemin OLTP, c'est le marché en direct. Il dépile cette file à une vitesse extrême pour croiser les ordres d'achat et de vente. Le chemin OLAP, c'est notre moteur de conformité (le gendarme financier). Il tourne en tâche de fond asynchrone et interroge directement des fenêtres glissantes en RAM pour ne jamais ralentir le fil d'exécution principal.

Mais encaisser techniquement la charge de données n'est que la moitié du problème.

Exactement. Car une fois que la donnée circule sans latence, il faut pouvoir repérer mathématiquement la fraude, et ce, en temps réel.

La méthode naïve consisterait à surveiller une simple formule heuristique : le *Cancel-to-Trade Ratio* (CTR). Ce ratio mesure le volume d'ordres annulés par rapport aux ordres placés. Si le système détecte qu'un acteur annule 95 % de ses ordres, il déclenche une alerte.

Le problème de cette approche statique, ce sont les faux positifs et les faux négatifs. Sur un marché de quotas carbone, un teneur de marché légitime (un *Market Maker*) peut naturellement annuler énormément d'ordres pour ajuster sa cotation face à la volatilité. À l'inverse, un Spoofer intelligent va passer quelques petits ordres d'achat parfaitement légitimes pour masquer sa signature et faire chuter son CTR global en dessous du seuil d'alerte.

Pour contourner ce biais, nous avons fait appel aux processus stochastiques en implémentant un Modèle de Markov Caché (HMM). La logique est la suivante : l'intention réelle du trader est une variable d'état cachée (est-il dans l'état « Légitime » ou l'état « Spoofer » ?). Le régulateur ne peut déduire cet état qu'à travers une séquence d'observations, c'est-à-dire les flux d'ordres émis.

Au lieu d'appliquer une règle rigide, notre algorithme, qui tourne sur le chemin OLAP, va évaluer à chaque microseconde la probabilité de basculement dans l'état fraudeur. Pour cela, le modèle pondère de manière asymétrique deux variables extraites de notre fenêtre glissante en mémoire vive :

1. L'historique global de l'acteur (le CTR).
2. Surtout, la *densité instantanée d'annulation* sur la toute dernière micro-fenêtre temporelle.

Si le spoofer tente de masquer son jeu avec du bruit, la concentration extrême de ses annulations au moment précis du leurre va mathématiquement saturer le calcul de densité. Le modèle probabiliste converge alors instantanément vers une certitude de fraude supérieure à 80 %, déclenchant l'alerte avant même que l'acteur n'ait eu le temps de se déconnecter du marché.

Partie Code :

Plongeons maintenant sous le capot pour comprendre comment ces concepts se traduisent concrètement dans notre code. Notre architecture est divisée en deux blocs totalement indépendants : un Backend (le cerveau financier) et un Frontend (le tableau de bord AMF)

Commençons par le cerveau, codé en Python avec le framework FastAPI. Le cœur de notre réacteur, c'est la gestion de l'asynchrone pour encaisser ce que l'on appelle les flux *Heavy Writes*. Au lieu de bloquer le serveur à chaque transaction, nous avons déclaré une simple file d'attente avec l'instruction `order_queue = asyncio.Queue()`. C'est elle qui applique le motif *Fan-In*. Toutes les requêtes entrantes y tombent instantanément.

En parallèle, notre fonction `market_engine` tourne en boucle infinie. Elle vient dépiler cette file à la vitesse de la mémoire vive pour croiser les ordres d'achat et de vente. C'est notre flux opérationnel, l'OLTP. Pour simuler la réalité d'un marché frénétique, nous avons codé des *tradings bots* autonomes qui tournent également en asynchrone et inondent cette file d'ordres aléatoires. Le serveur encaisse la charge sans jamais monopoliser la puissance de calcul.

Mais une fois l'ordre traité, le `market_engine` ne s'arrête pas pour faire des mathématiques. Il déclenche immédiatement notre moteur analytique en tâche de fond, séparant ainsi strictement l'OLTP de l'OLAP.

C'est ici qu'intervient le flux analytique (l'OLAP) avec notre fonction `compliance_engine`. C'est elle qui embarque notre modèle de Markov Caché (HMM) pour la détection du spoofing.

Pour que ce calcul soit exécuté en temps réel, nous faisons du *Complex Event Processing (CEP)*. Nous n'interrogeons aucune base de données lente. Nous analysons l'historique du trader directement dans la mémoire RAM via des structures optimisées en C (les `collections.deque`).

Le code extrait d'abord une métrique de fond : le *Cancel-to-Trade Ratio* (CTR). Mais surtout, il isole les toutes dernières microsecondes d'action du trader pour calculer la variable décisive : la densité instantanée de retrait.

La probabilité mathématique de l'état caché $P(S=\{\text{Spoofers}\})$ est ensuite calculée en fusionnant ces deux dimensions, auxquelles nous avons injecté une variance stochastique pour simuler un modèle organique et non binaire. Dès que la probabilité dépasse le seuil critique de 80 %, l'algorithme génère un flag de fraude.

Il fallait enfin une interface pour visualiser ces alertes. Nous avons développé un tableau de bord avec Streamlit, qui représente la salle de contrôle des régulateurs.

La règle d'or de ce Frontend, c'est qu'il est complètement « aveugle ». Il ne fait aucun calcul métier, aucune modélisation. Il utilise une simple fonction utilitaire `fetch_data` avec un bloc de sécurité `try/except`. Son seul rôle est de frapper à la porte de notre API pour récupérer l'état du carnet d'ordres et les alertes HMM. Si le serveur HFT venait à saturer, l'interface renverrait simplement une valeur nulle au lieu de crasher violemment.

Pour donner l'illusion parfaite du temps réel, nous utilisons une boucle de rafraîchissement en bas de notre script avec les commandes `time.sleep()` et `st.rerun()`. Chaque seconde, la page se relance automatiquement, actualise les graphiques et fait clignoter les fraudes détectées.

Partie Conclusion :

Cependant, notre système actuel reste un prototype académique qui possède ses limites. Pour le transformer en un véritable outil industriel capable de surveiller des flux boursiers mondiaux, plusieurs évolutions seraient indispensables.

Tout d'abord, au niveau de l'infrastructure pure. Actuellement, notre application tourne localement et le journal d'audit est géré par une base de données SQLite. Pour passer véritablement à l'échelle, il faudrait déployer cette architecture dans le Cloud. Nous devrions remplacer notre file d'attente native Python par un outil de traitement de flux distribué massif, comme Apache Kafka. De même, pour la persistance de la donnée, il faudrait migrer vers une base de données NoSQL distribuée, comme Cassandra ou MongoDB, pour garantir la haute disponibilité en cas de panne matérielle.

Ensuite, du côté de l'intelligence artificielle, il y a également une grande marge de progression. Notre modèle de Markov Caché intercepte très bien le spoofing classique. Mais face à des fraudeurs qui utilisent eux-mêmes des intelligences artificielles pour muter et masquer leurs traces, il serait nécessaire d'intégrer des modèles de Machine Learning plus profonds (comme le Deep Learning). Cela permettrait au système de surveillance d'apprendre automatiquement les nouvelles signatures de manipulation au fil du temps.